Arceus

A Startup that creates startups

Introduction

- A Startup is a temporal, evolving entity that solves a given problem P given the industrial, technological, emotional constraints.
- A startup creation process is a pattern , trajectory itself that somehow attempts to solve the problem.
- Today dynamics are : 10(employees)*1000(companies)
- With advancement in LLMs, their training technique and also mathematics, we can bring this dynamics to <5(employees)*large_number(company)
- A company basically solves a problem which is a pattern and patterns are infinite.

Why Arceus?

- The field of Artificial Intelligence has evolved to a stage that technically every individual just ideates on the system, figures out what to build and builds it using AI.
- A year ago, AI was used to do grunt work but today it has evolved with coming of models like GPT-5.4 and Opus-4.6, it is able to design a system about a given problem and code it too end-to-end.
- Sooner this idea will be worked by multiple folks but we will be the pioneers and what differentiates us : We will use both mathematics and engineering to solve it.
- Arceus solves two problems:
  - What to build?
  - Encompassing the dynamics of the startup process so that User just delegates like Board of Director

Who?

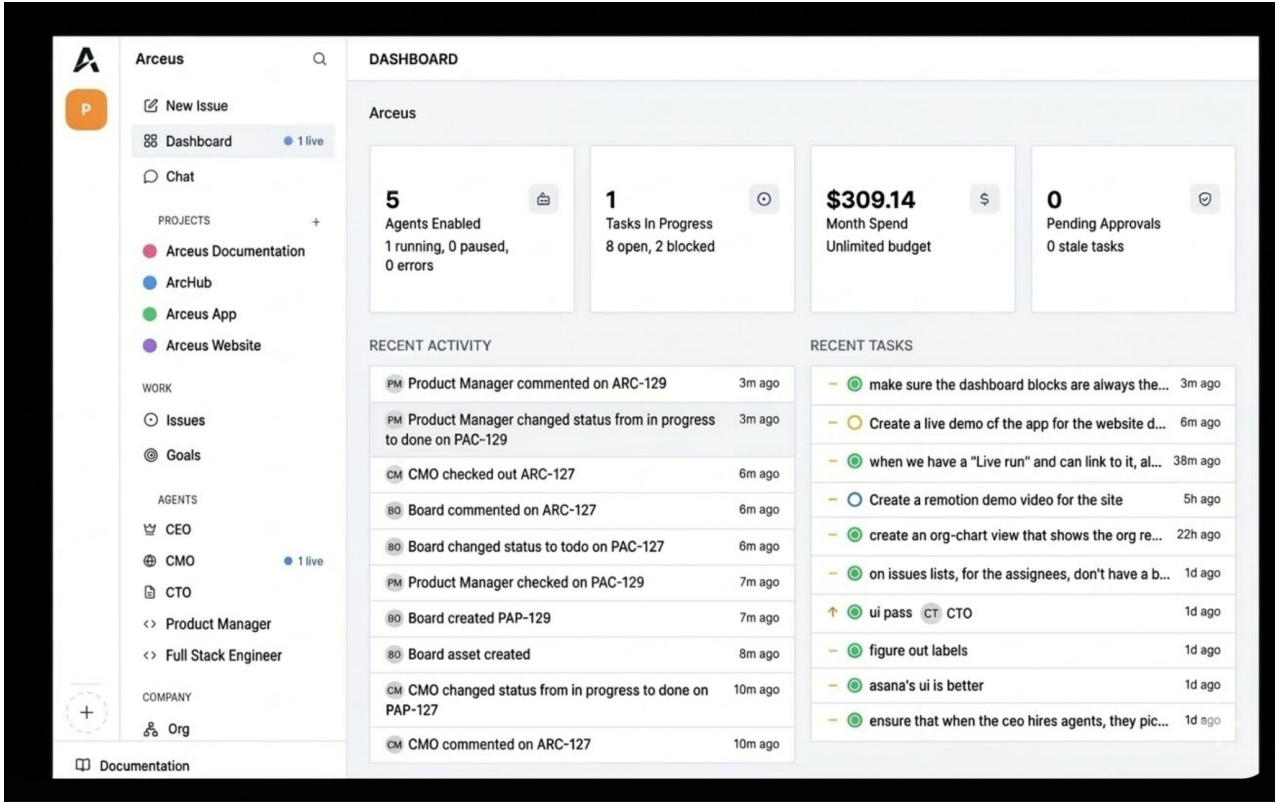
- People who have ideas / patterns but don’t have resources or have friction to build.
- Folks who enjoy working solo but want to build something.
- Someone who wants to work on a problem and do a specific skill(coding,design etc) and don’t want to hire a team for other stuff.

Dynamics of a Startup

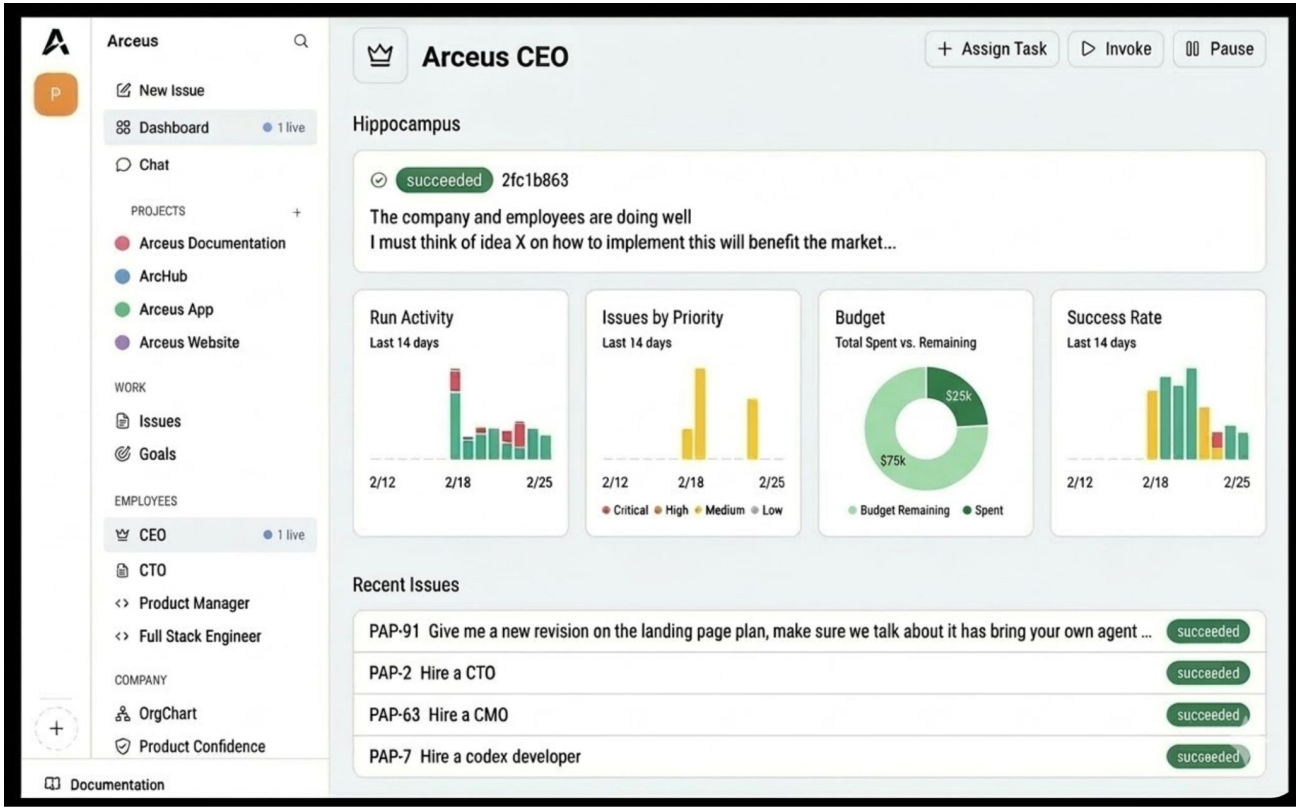
- There are Founders Fs, Employees Es, Investor.
- Investor is a person that gives capital, believes in the idea, help the startup grow by giving direction.
- Founders are the folks that bring about the idea , are technically the first employees and maybe CEO,CTO etc
- Then we have specialized employees; for instance PMs, Full-stack Engineers, ML Researchers etc
- CEO and CTO navigate the ship backed and guided by the investors; we have syncs where employees interact and align themselves with the problem.
- Every employee have their own belief system + skillset that helps to build the startup as it is today.
- Every day people talk, share ideas, research the trends and tailor their startup according to the industry trends.

Example Dashboard Designs

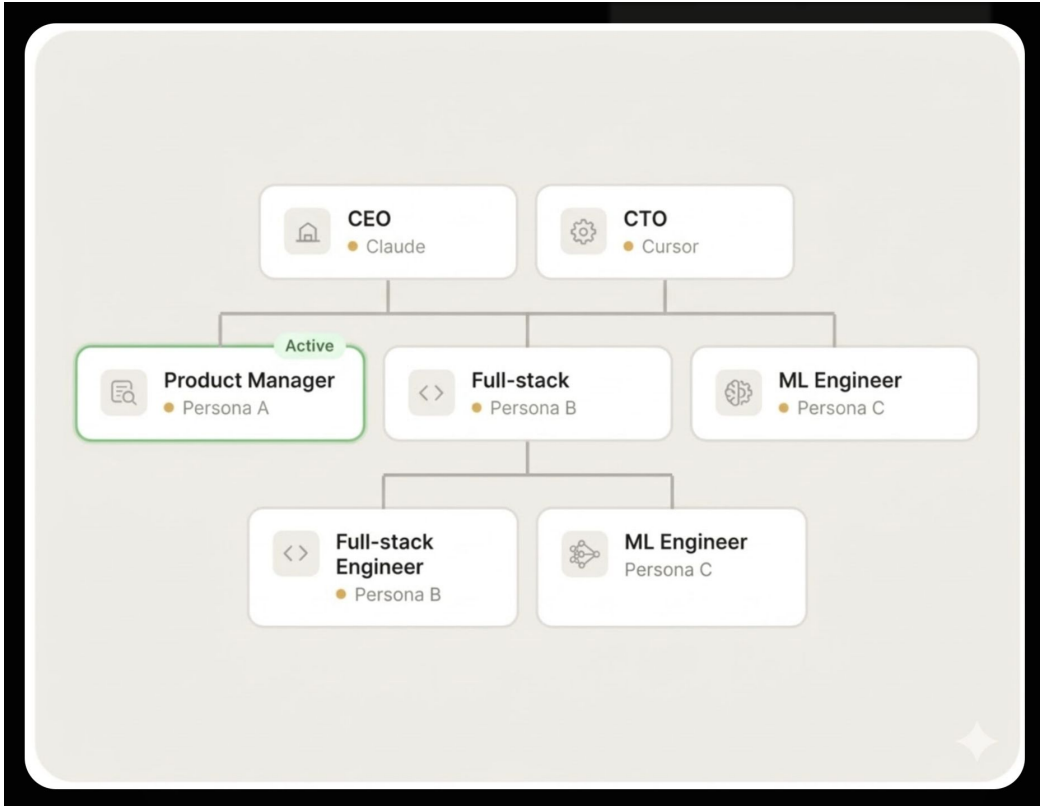
FrontPage



Employee Page



Org Hierarchy



Ticket Log

Ticket System

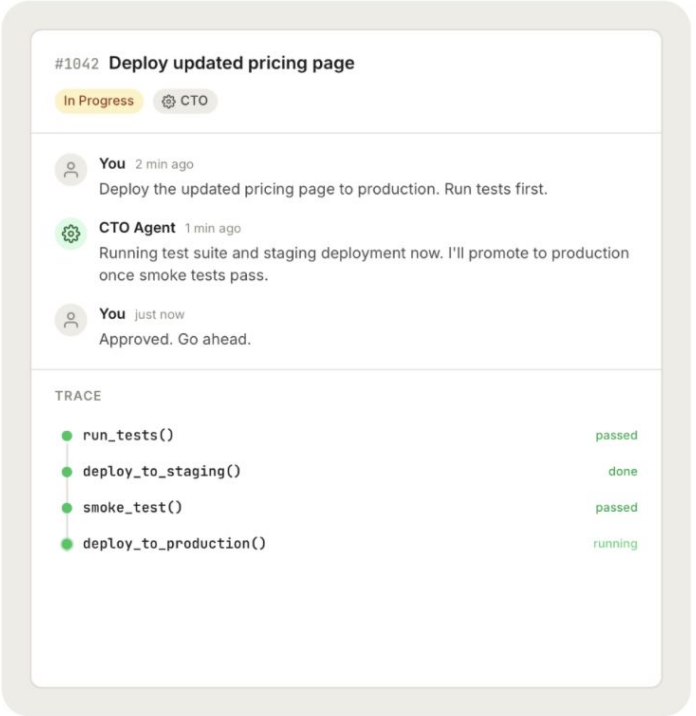
Every conversation traced. Every decision explained.

You communicate with agents through tickets. Every instruction, every response, every tool call and decision is recorded with full tracing. Nothing happens in the dark.

Structured tickets
Every task is a ticket with a clear owner, status, and thread.

Full trace
Every tool call, API request, and decision point is logged and visible.

Immutable audit log
Append-only history. No edits, no deletions. Full accountability.



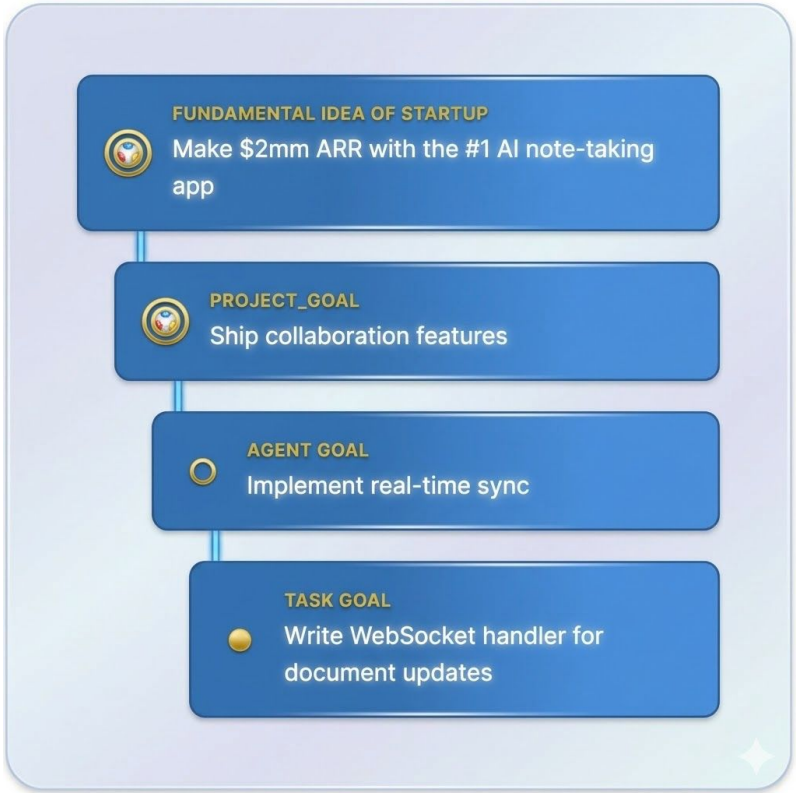
Idea of Startup

- Every startup always starts with a core idea.
- Ideas are pivoted considering market research, potential value of the idea, any specific nuance of the core idea that gets popularity.
- Assumption: Even though a startup pivots its idea, the core idea(supreme) always remain.
- For example:
    - Slack:
        - Game with in-chat feature; in-chat became famous and it is slack today
            - A interface where people could meet for **strategic thinking**.
    - Instagram started off with people sharing todo lists, photo sharing and pivoted to photos and reel as that was profitable
        - Core idea: A platform where 2 users interact with **activity**.

High Level Imagination

- Internally: We will have agents encompassing CEO,CTO, PM, Full-stack etc
- Every employee-agent will have its own memory(Hippocampus) + its ability to create sub-agents to solve Tasks.
- User has to give a preliminary Budget B(dynamically calculated) which will be the minimum Budget to create an MVP for their idea.
- A Dashboard D will be provided for the user where Idea is refined with the user, a team is built(maybe starting only with CEO & CTO)
- The Dashboard will show the user progress on every version of the startup, the employee working metrics, resources, tasks assigned to them.
- He will also have first principle breakdown of all problems, sub-problems and can give his input wherever he wants.

Breakdown Based on First Principles



Meeting

- We will make a construct of meeting where all Employees will update whatever they have done in broad goals.
- CEO gives feedback from user, its own research in the meeting for company vision.
- Any pain point problem like less hitrate for a problem solved will have the discussions and will be stored as “Learnings” construct for that employee.
- Positive affirmations for employees will be delegated so it gets stored in memory and helps while running the system
- User will have a meeting dashboard where he can see meeting notes, sync task done by agents.
- Task list will be modified according to meetings

Pivot

- Pivot will come post MVP. The initial shift will be handled by the refine-ment done by User and CEO.
- In a Pivot, either the User asks the CEO to shift direction or The CEO decides on its own research(less weightage to this in initial version).
- Pivot will be a special meeting construct where every Employee will be told the new domain.
- Pivot is an important concept for our design as it will alter the memory, skills
- To what extent , do we update/delete our existing memory has to be decided by the Pivot class.

First Principles Approach

- The User will give the problem and after refinement , we will derive the core value problem of the User.
- For MVP, The core Idea is the fundamental idea for the startup.
- All employees will encompass the fundamental Idea.
- CEO,CTO will consider core value as the fundamental truth for them and they can pivot the fundamental idea along with user feedback.
- User is the Board of Director of startup, looking at vision, journey and also resource management.
- Every employee delegates the direction of its lower employee.
- Every employee has the ability of Self-spawning Orchestration system and their own Hippocampus(Memory).

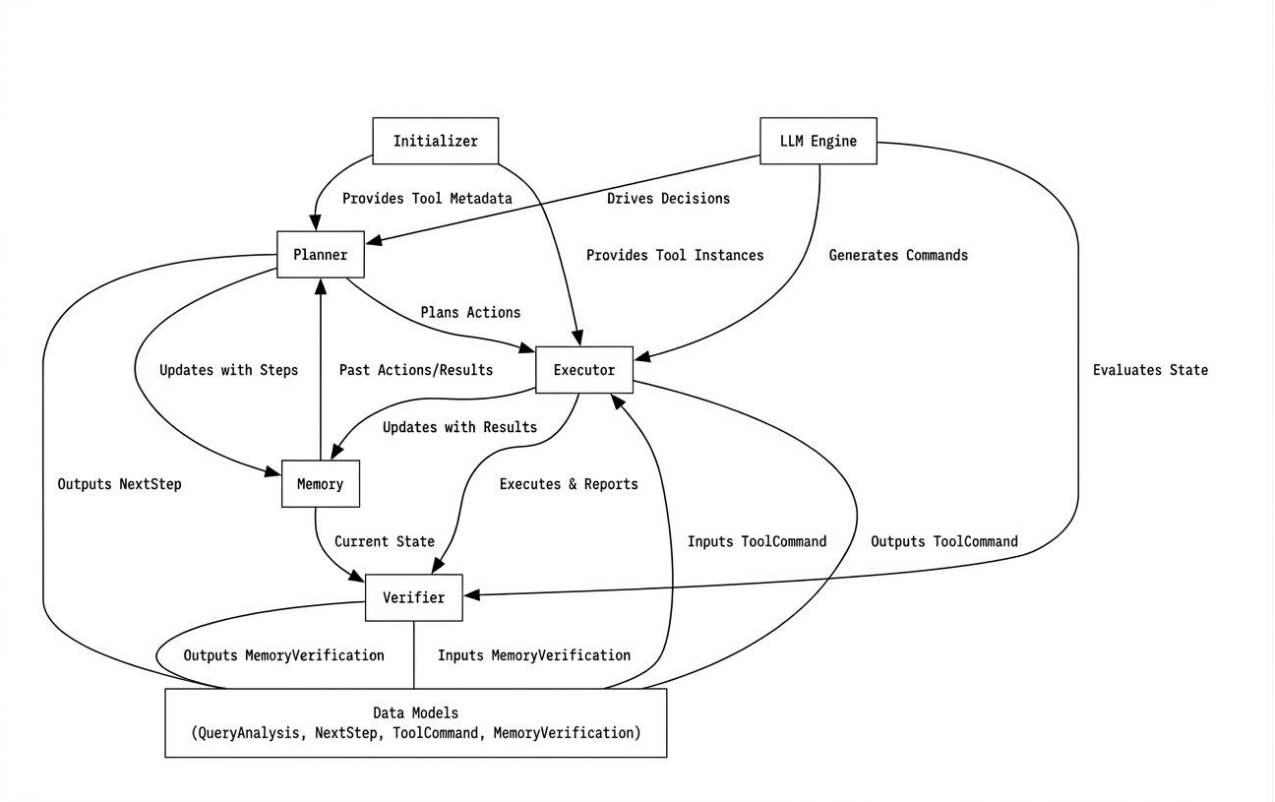
Task - First principles for Self- Spawning Architecture

- This is the most fundamental unit of the entire startup. Every agent(employee) and sub-agent(spawned entity) will have a Task.

- Every problem will be broken down into sub-problems using First Principle Thinking and consensus of all of the sub-problems broken will be the solution of a problem.

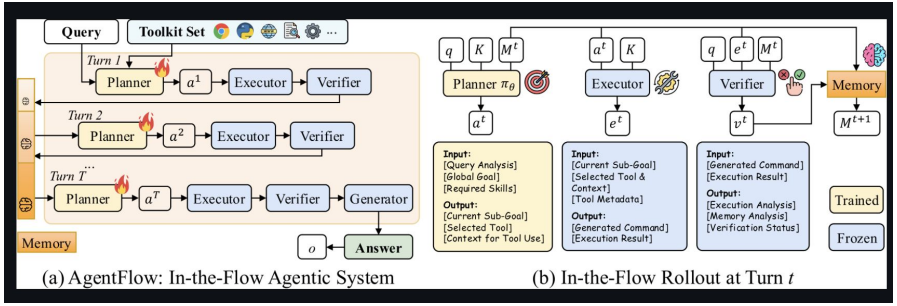
- Task is made up of a planner, executor, verifier, Memory(Hippocampus) and any reusable class blueprint that maybe required.

Task Design


Initializer
LLM Engine
Provides Tool Metadata
Drives Decisions
Planner
Provides Tool Instances
Generates Commands
Plans Actions
Updates with Steps
Past Actions/Results
Executor
Evaluates State
Updates with Results
Outputs NextStep
Memory
Executes & Reports
Current State
Inputs ToolCommand
Outputs ToolCommand
Verifier
Outputs MemoryVerification
Inputs MemoryVerification
Data Models
(QueryAnalysis, NextStep, ToolCommand, MemoryVerification)

Task Design - RL based Training For sub-agents

- LLMs with good context engineering is really good at solving a single problem but when we give it a trajectory of steps to do , it hallucinates.
- The reason people are investing in multiple agent orchestration as you need to optimise for trajectory of each agent and somehow make the system also work.
- For sub-agents, we can take SLM like Qwen3 and train it on making effective plans for Trajectory using GRPO, we can make basically every spawned entity good enough in itself to be a good agent for the task it is assigned.

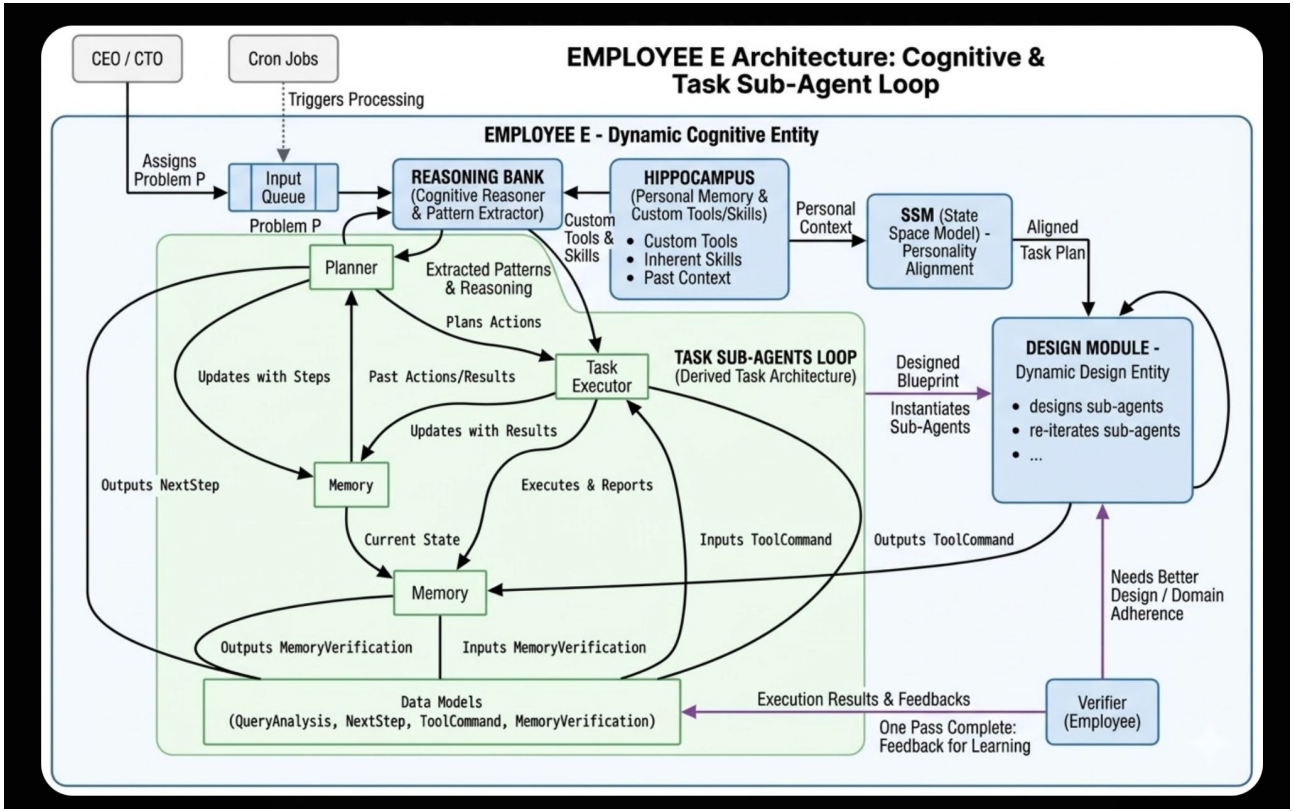


(a) AgentFlow: In-the-Flow Agentic System (b) In-the-Flow Rollout at Turn t

Sub-Agents Types

- Generic Agents
  - They are ones using the RL Task Framework.
  - These agents do any task given the problem, tools and skills present in the directory of the Employee.
- Specialized Agents
  - They are specialised agent for a particular task for example: BrowserUse, Code Generation and Execution
  - They could be LLMs or LLMs with sub-network of RL Trained subagents.
- Exploratory Agents
  - These agents are the ones that reasons out, researches on problem, upskilling etc(intelligent spawning).
  - They are also called when both the two agents couldn’t do a task and they have to think from scratch.

Employee Design



HippoCampus

- There will be a specialized Framework dealing with Hippocampus called as **ReasoningBank.**
- Memory will be multi-tier i.e. the level above has some control over the memory of level below(can decide updation/deletion in memory).
- Every Employee(agent) delegates what chunk of memory has to be assigned to a sub-agent for overall context.
- Memory is divided into Different levels and each of them have their own use - case.
- Hippocampus models in Memory, Patterns and State of the Employee.

Memory

Types of Memory:

- In-memory
  - This is the working Memory of the Agent(the run-time logs). Every agent and sub-agent will have their own In-Memory which they have full control over.
- Long-Term Memory(Conscious)
  - Semantic Memory
    - General Knowledge of the world(Semantic Retrieval of Knowledge)
  - Episodic Memory
    - Memory of the event you are doing(Doing a coding task and you need knowledge of repo)
- Long-Term Memory(Unconscious)
  - Priming Memory
    - A change in response to a stimulus due to prior experience to it(State of the Employee)
  - Procedural Memory
    - Responsible for knowing how to perform actions(Habits)

Pattern

- Patterns will be the Construct through which Memory will be used , either semantic or episodic.
- Consider Memory is something stored deep inside a brain and while you do a task, you process it as a pattern to do that task.
- ReasoningBank retrieves memory, converts them in pattern to be used by the agents and evolve , delete, prune, merge different patterns whenever redundant patterns appear.
- A faith of a pattern is decided by its quality score which could be evaluated by LLM as a judge or some heuristic way.

Habit

- Just like humans learn and do some things out of habit, these agents must also be evolving as time goes.
- Habit are blocks in system Prompt of the agent that are given explicit instructions to get triggered immediately when a condition is met.
- Just a replication of Procedural Memory.
- New idea: Have to think this in depth.

ReasoningBank

- One of the parts of the Hippocampus that helps how to process memories.
- ReasoningBank will be unique for each employee and this will delegate memory to pass to the sub-agents along with their own Working Memory.
- Framework for ReasoningBank:
  - Retrieve: Retrieves Top-k memory units according to the retrieval algo.
  - Judge: Judges an Agent’s Trajectory and given the top-k retrieved memory, decides to keep it in Memory or not
  - Distill: Distills the agent trajectory as proper memory unit . Also used for creating patterns.
  - Consolidate: When this is triggered the Employees memory store gets updated by new learnings etc.

Consolidate - Self-Learning Cycle of Employee

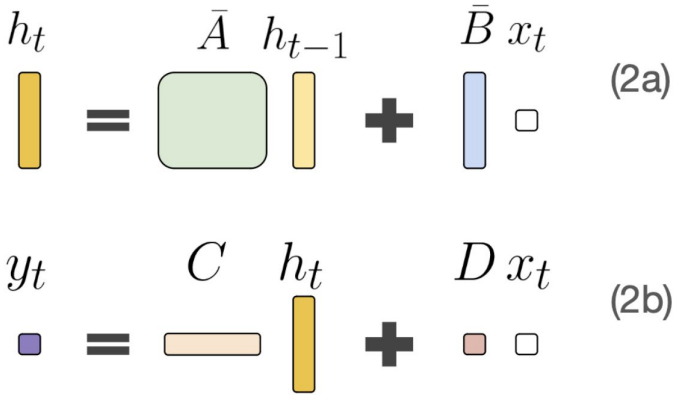
- Whenever you trigger consolidation after retrieve, judge and distill, following steps happen:
  - Patterns evolve, get merged or pruned.
  - Memory DB gets updated(duplicated memories are deleted)
  - Skills get an update or a new skill is created considering various factors of a patterns, its reusability/quality score.
  - A Habit is formed if any memory block, skills, pattern is being used a lot.
  - Consolidate can also be used for evolving tools(exploratory agent gets a new tool)
- Basically consolidation is a self-learning framework of the Agent that helps it to adapt to the Task at hand.
- We can use continual learning/RL algo for quality score computation and pattern mechanics(To think)

Domain Driven Distillation(Experimental)

- Let’s say I get a Trajectory T which has to be distilled as a Memory Unit.
- Although the LLMs are themselves good at adaptive learning , but to give a fine edge we could incorporate domain driven distillation of a Trajectory.
- This approach will be used when our platform makes a MVP level of architecture as it needs context.
- There will be feedback level at every sub-agent level that takes in how efficiently it follows instruction of employee and how much aligned it is to the Fundamental Idea of startup
- After collection of dataset, we will trigger continual training of a SLM using RL that given a text , it aligns it to the Task based on the dataset we have.
- So during Distillation we then store distilled memory as domain driven units.

State Space Models and Mamba

- A is the transition state matrix. It shows how you transition the current state into the next state. It asks "How should I forget the less relevant parts of the state over time?"
- B is mapping the new input into the state, asking "What part of my new input should I remember?"[11]
- C is mapping the state to the output of the SSM. It asks, "How can I use the state to make a good next prediction?"[12]
- D is how the new input passes through to the output. It's a kind of modified skip connection that asks "How can I use the new input in my prediction?"



Visual Representation of The SSM Equations

Selectivity allows each token to be transformed into the state in a way that is unique to its own needs. Selectivity is what takes us from vanilla SSM models (applying the same A (forgetting) and B (remembering) matrices to every input) to Mamba, the ***Selective*** *State Space Model.*

In regular SSMs, A, B, C and D are learned matrices - that is

$\mathbf{A} = \mathbf{A}_\theta$ etc. (where θ represents the learned parameters)

With the Selection Mechanism in Mamba, A, B, C and D are also functions of x. That is $\mathbf{A} = \mathbf{A}_{\theta(x)}$ etc; the matrices are context dependent rather than static.

Algorithm 1 SSM (S4)

Input: x : (B, L, D)
Output: y : (B, L, D)
1: A : (D, N) ← Parameter
▷ Represents structured $N \times N$ matrix
2: B : (D, N) ← Parameter
3: C : (D, N) ← Parameter
4: Δ : (D) ← τ_Δ(Parameter)
5: $\overline{A}, \overline{B}$: (D, N) ← discretize(Δ, A, B)
6: $y \leftarrow \mathrm{SSM}(\overline{A}, \overline{B}, C)(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: x : (B, L, D)
Output: y : (B, L, D)
1: A : (D, N) ← Parameter
▷ Represents structured $N \times N$ matrix
2: B : (B, L, N) ← $s_B(x)$
3: C : (B, L, N) ← $s_C(x)$
4: Δ : (B, L, D) ← τ_Δ(Parameter+$s_\Delta(x)$)
5: $\overline{A}, \overline{B}$: (B, L, D, N) ← discretize(Δ, A, B)
6: $y \leftarrow \mathrm{SSM}(\overline{A}, \overline{B}, C)(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

Mamba (right) differs from traditional SSMs by allowing A,B,C matrices to be **selective** i.e. context dependent (source)

Incorporating Mamba(Experimental)

- While you are transferring the instructions from Reasoning Bank to a Sub-agent , you pass it through this Mamba block.
- The Mamba block basically aligns the instructions to the Employee’s state.
- Artificially simulate a state for an employee and pre-train it with Mamba to get learnable weights.
- Use this pre-trained Mamba to align with the Employee values, vision and personality.
- This step gives personality to the employee.
- Can give customizable widgets for personality of that particular employee and you can make it adapt to that personality.